

# Malware Behavior Comparison: Remote Access Trojan, Fileless, and Living-off-the-Land

Soleil Cordray

hi@solcor.dev

University of Central Florida

Orlando, Florida, USA

## Abstract

This paper presents a comparative static analysis of three malware categories: Remcos Remote Access Trojan (RAT), fileless malware, and living-off-the-land (LotL) attacks. Remcos RAT analysis utilized the Ghidra reverse engineering framework, while fileless and LotL categories were analyzed directly for comparison. The goal was to develop a classification methodology that identifies the behavioral patterns that distinguish each category while noting shared techniques.

Analysis of a real Remcos RAT sample revealed registry-based persistence through `RegSetValueExA`, dynamic API loading to evade import analysis, and an embedded C2 URL (`dsoft1961.narod.ru`). Educational pseudocode samples for fileless and LotL categories were developed based on MITRE ATT&CK and LOLBAS Project techniques to enable comparison without handling live malware. The results include a behavioral classification table, an Indicators of Compromise (IOC) catalog, and annotated Ghidra screenshots. The analysis shows that while all three categories implement persistence mechanisms, they differ significantly in execution strategy: RATs use compiled binaries with obfuscation, fileless malware operates through scripting engines in memory, and LotL attacks chain trusted Windows utilities.

**Code & Artifacts:** <https://github.com/soleil-cordray/malware-behavior-comparison>

## 1 Introduction and Problem Statement

Modern malware has evolved beyond simple executable files that antivirus can easily identify. Three categories give defenders particular difficulty: Remote Access Trojans (RATs) that enable remote system control, fileless malware that runs entirely in memory without dropping files, and living-off-the-land (LotL) attacks that abuse legitimate Windows tools [9, 10]. These threats are difficult to detect because they hide in plain sight using trusted processes, avoid obvious artifacts, and blend into normal system activity [3].

Remcos RAT is commercially sold as a "remote administration tool" but is widely used for credential theft, keylogging, and maintaining persistent access to compromised systems [8]. Security researchers have documented its use by multiple threat actors in phishing campaigns targeting financial institutions, government agencies, and healthcare organizations. The malware supports extensive functionality including screen capture, webcam access, microphone recording, and file system manipulation. Fileless malware takes a different approach, using PowerShell or other scripting engines to run malicious code directly in memory without writing executable files to disk [3]. This architectural choice makes traditional file-based scanning inherently ineffective, since there are no

malicious files to detect. LotL attacks make detection even harder by using Microsoft-signed tools like `certutil`, `mshta`, and `bitsadmin` to download, execute, and exfiltrate data [9]. Blocking these tools is impractical because they serve legitimate administrative purposes; security teams must instead monitor suspicious usage patterns.

This project classifies these malware types through static analysis using Ghidra [11] and direct code observation. The goal is to develop a methodology that identifies what makes each category distinct while noting shared techniques. By analyzing a real Remcos sample alongside educational pseudocode samples, the project extracts IOCs and builds a comparison framework applicable to other malware families. The deliverables include annotated Ghidra analysis with screenshots, a behavioral classification table mapping techniques to MITRE ATT&CK identifiers, and a comprehensive catalog of indicators useful for detection and response.

## 2 Related Work

The LOLBAS Project maintains a comprehensive database documenting abuse patterns for more than 150 legitimate Windows binaries, including `certutil`, `mshta`, `rundll32`, `bitsadmin`, and `regsvr32` [9]. Each entry documents the binary's legitimate purpose, abuse methods, and detection opportunities. MITRE ATT&CK provides systematic documentation of adversary techniques including signed binary proxy execution (T1218), PowerShell abuse (T1059.001), ingress tool transfer (T1105), and boot/logon auto-start execution (T1547.001) [10]. These resources informed the development of the sample and its behavioral classification.

Industry threat intelligence provides operational context. Elastic Security Labs identified Remcos RAT's Delphi-based architecture, encrypted configuration storage, and dynamic API resolution to evade import table analysis [8]. Their multi-part analysis series documented specific function addresses responsible for C2 communication, keylogging, and credential theft. CrowdStrike documents PowerShell's role in fileless attacks, including Base64-encoded execution and `Invoke-Expression` for in-memory payload delivery [3]. The Center for Internet Security documents real-world LotL campaigns demonstrating how attackers chain trusted binaries for complete attack life-cycles without deploying custom malware [5]. Existing ANY.RUN sandbox reports provide dynamic analysis baselines showing Remcos runtime behaviors that include registry modifications, process injection, and network callbacks [2].

Ghidra methodologies are well-documented in security research. Varonis provides workflows that emphasize import analysis, string extraction, and cross-reference tracing as the foundational steps towards understanding unknown binaries [7]. Hack The Box outlines the fundamentals of static analysis, including

suspicious API identification and behavior reconstruction from disassembly [6]. A foundational text written by Sikorski and Honig, *Practical Malware Analysis* [12], established safe handling practices that informed the isolated VM approach used in this project. Public proof-of-concept implementations such as the `bytecode77` repository [4] demonstrate practical fileless attack chains combining PowerShell execution with registry persistence, directly informing the development of this comparison’s educational sample. The NSA’s release of Ghidra as open-source software [11] democratized access to professional-grade reverse engineering capabilities previously limited to expensive commercial tools.

### 3 Methodology

This project follows three phases: environment configuration, sample acquisition, and static analysis. Figure 1 illustrates the analysis pipeline. All work was conducted within an isolated virtual machine.

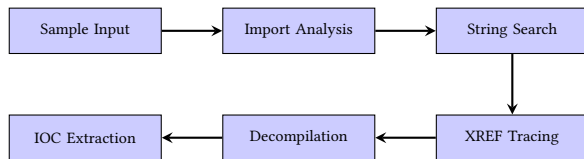


Figure 1: Static analysis pipeline for IOC extraction.

#### 3.1 Analysis Environment

A Windows 10 Pro VM was deployed in VMware Fusion on a 2019 Intel MacBook Pro. Network isolation was configured through VMware’s “Private to my Mac” setting [13], critical when handling live malware. Windows Defender was disabled to prevent sample quarantine. Multiple VM snapshots enabled rapid restoration between sessions. Ghidra 11.0 with OpenJDK 21 [11], Notepad++, 7-Zip, and HxD were installed.

#### 3.2 Sample Acquisition and Development

Three samples were prepared (Table 1). A real Remcos RAT sample was obtained from MalwareBazaar [1] by searching for the tag “RemcosRAT”. The SHA256 hash is recorded in the GitHub repository for reproducibility. To avoid handling live malware, fileless and LotL categories are represented by educational pseudocode developed from documented techniques [4, 9].

Table 1: Sample Summary

| Category   | Type        | Source        | Tool      |
|------------|-------------|---------------|-----------|
| Remcos RAT | Real binary | MalwareBazaar | Ghidra    |
| Fileless   | Pseudocode  | Custom        | Notepad++ |
| LotL       | Pseudocode  | Custom        | Notepad++ |

The **Fileless Loader** pseudocode documents seven stages: (1) error suppression via `$ErrorActionPreference`, (2) anti-sandbox

environment checks comparing hostnames against analysis patterns, (3) Base64-encoded C2 communication via `System.Net.WebClient` with browser user-agent spoofing, (4) payload decoding via `FromBase64String`, (5) in-memory execution via `Invoke-Expression`, (6) registry persistence through the Run key, and (7) system information exfiltration. Each stage maps to documented MITRE ATT&CK identifiers.

The **LotL sample** documents abuse of nine Windows utilities from LOLBAS [9]: PowerShell for Defender disabling (T1562.001), `certutil` for payload download (T1105), `mshta` for script execution (T1218.005), `schtasks` for scheduled task persistence (T1053.005), `rundll32` for LSASS credential dumping (T1003.001), `reg` for firewall manipulation (T1562.004), `bitsadmin` for exfiltration via BITS transfers (T1048), `wevtutil` for Windows event log clearing (T1070.001), and `regsvr32` for Squiblydoo script execution (T1218.010).

#### 3.3 Ghidra Analysis Workflow

Remcos analysis followed six steps based on established methodology [6, 7]: (1) project setup with auto-analysis (3-4 minutes for the 460KB sample), (2) entry point examination via Function Graph (Figure 2), (3) import table analysis for capability assessment, (4) string search with filters for “http”, “Run”, and “Software”, (5) cross-reference tracing to understand API usage context, and (6) IOC documentation throughout analysis.

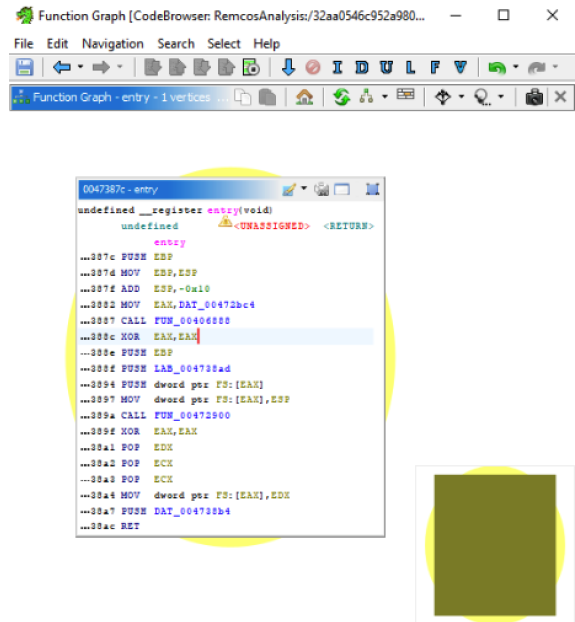


Figure 2: Function graph of Remcos entry point showing initial program flow.

4 Evaluation and Results

4.1 Remcos RAT Binary Analysis

Static analysis of the Remcos sample in Ghidra revealed four significant findings regarding its capabilities and evasion techniques.

**Finding 1: Dynamic API Load for Evasion.** The import table (Figure 3) shows ADVAPI32.DLL, COMCTL32.DLL, GDI32.DLL, KERNEL32.DLL, MSIMG32.DLL, OLEAUT32.DLL, SHELL32.DLL, USER32.DLL, VERSION.DLL, and WINMM.DLL. Notably absent are WS2\_32.DLL (Winsock) and WININET.DLL (Internet APIs), which would be expected for a RAT with network capabilities. Since Remcos is documented to have C2 functionality, this absence suggests that the sample resolves networking APIs dynamically at runtime via GetProcAddress—MITRE ATT&CK technique T1027.007 (Dynamic API Resolution) [8].

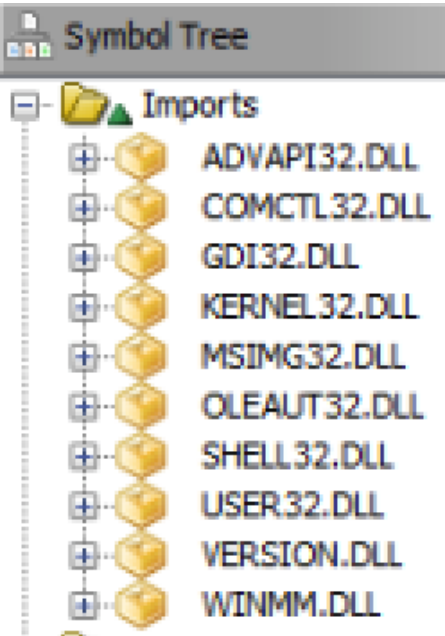


Figure 3: Import table lacking networking libraries, indicating dynamic API resolution.

**Finding 2: Registry Persistence APIs.** Expanding ADVAPI32.DLL in the imports revealed registry manipulation functions: RegCloseKey, RegCreateKeyExA, RegFlushKey, RegOpenKeyExA, RegQueryValueExA, and RegSetValueExA (Figure 4). These APIs are commonly used for persistence via the Registry Run key (T1547.001). The complete registry API suite—open, create, query, set, flush, close—indicates sophisticated manipulation capability beyond simple persistence.

Cross-reference analysis of RegSetValueExA revealed three call locations at the address 0x00406a18 (Figure 5). Tracing these XREFs showed that the function was invoked in contexts consistent with persistence establishment. Although the specific registry path was not visible in plaintext (likely encrypted), the API usage pattern strongly suggests Run key persistence [8].

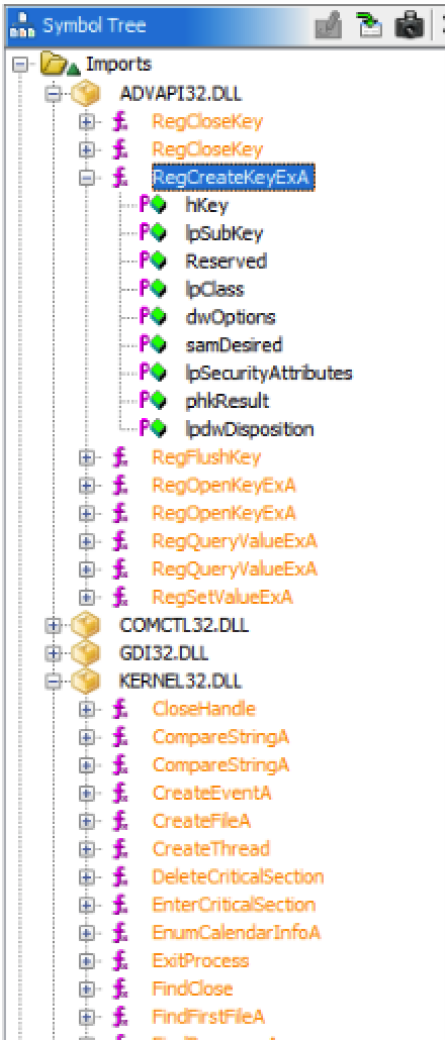


Figure 4: ADVAPI32.DLL imports showing registry manipulation APIs.

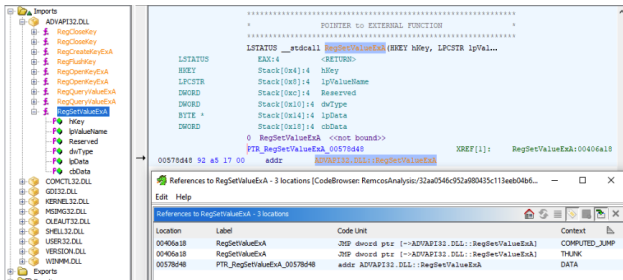


Figure 5: Cross-references to RegSetValueExA showing three call locations.

**Finding 3: Embedded C2 URL.** String analysis using Search → For Strings with filter "http" revealed an embedded command-and-control URL: http://dsoft1961.narod.ru at the

address 0x004712c0 (Figure 6). Cross-referencing demonstrated its use by FUN\_00470f18, likely the network initialization routine. The use of a Russian free hosting service is consistent with Remcos infrastructure documentation [2].



Figure 6: Embedded C2 URL with cross-reference to calling function.

**Finding 4: Delphi Compiler Artifacts.** String analysis revealed Borland Delphi compiler signatures (Figure 7), including "Software\Borland\Delphi\Locales" and "Software\Soft\Fifteen". This corroborates Remcos' documented architecture [8] and provides additional context for the characteristic import patterns observed in Finding 1.

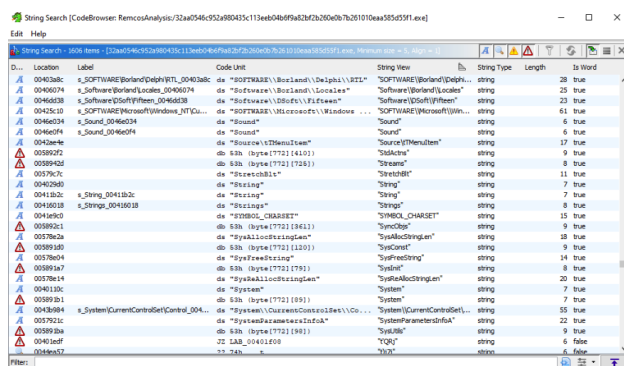


Figure 7: Delphi compiler artifacts in string analysis.

## 4.2 Fileless and LotL Analysis

The fileless pseudocode documents techniques mapped to MITRE ATT&CK: error suppression (T1562), reconnaissance via environment variables (T1082), C2 configuration with WebClient (T1071), payload download without disk writes (T1105), Base64 decoding (T1140), memory execution via Invoke-Expression (T1059.001), registry persistence (T1547.001), and exfiltration (T1041). The key characteristic distinguishing fileless malware is the complete absence of malicious files on disk; all execution occurs within the PowerShell process memory space, making traditional antivirus file scanning ineffective.

The LotL pseudocode demonstrates a complete attack chain using Microsoft-signed binaries (Table 2). Defense evasion occurs first. Disabling Defender ensures that subsequent stages continue uninterrupted. The certutil abuse is particularly effective because

it is a legitimate certificate management tool; network security tools often whitelist its traffic patterns. Persistence via schtasks offers significant advantages over registry-based persistence: administrators can specify privilege levels (HIGHEST), configure flexible triggers (ONLOGON, ONSTART, specific times), and scheduled tasks are often harder to enumerate than well-known registry locations. The credential dumping stage via rundll32 calling comsvcs.dll MiniDump is a well-documented technique for LSASS memory extraction that can capture plaintext passwords and NTLM hashes. Finally, evidence destruction via wevtutil clearing Security and System logs significantly complicates incident response and forensic investigation.

Table 2: LotL Attack Chain—Abused Windows Utilities

| Binary     | Abuse Purpose         | ATT&CK    |
|------------|-----------------------|-----------|
| powershell | Disable Defender      | T1562.001 |
| certutil   | Download payload      | T1105     |
| mshta      | Execute VBScript      | T1218.005 |
| schtasks   | Create scheduled task | T1053.005 |
| rundll32   | Dump credentials      | T1003.001 |
| reg        | Manipulate firewall   | T1562.004 |
| regsvr32   | Squiblydoo execution  | T1218.010 |
| bitsadmin  | Exfiltrate via BITS   | T1048.003 |
| wevtutil   | Clear event logs      | T1070.001 |

## 4.3 Comparative Classification

Table 3 compares categorized behavioral patterns. Persistence shows convergence: all categories require reboot survival. Execution and evasion diverge: Remcos has built-in obfuscated capabilities, fileless depends on scripting engines, and LotL requires tool chaining. The C2 row shows different operational choices: direct HTTP, encrypted WebClient, or BITS transfers. Detection requires category-specific approaches.

Table 3: Behavioral Classification Matrix

| Behavior    | Remcos          | Fileless       | LotL            |
|-------------|-----------------|----------------|-----------------|
| Persistence | Registry APIs   | Registry (PS)  | Scheduled tasks |
| Execution   | Compiled PE     | PowerShell IEX | Trusted bins    |
| Evasion     | Dynamic APIs    | Memory-only    | Signed bins     |
| C2 Channel  | HTTP            | HTTPS          | BITS            |
| Detection   | Import analysis | Script logging | Cmd audit       |

## 4.4 Cross-Sample Code Comparison

**Persistence:** Remcos uses RegCreateKeyExA / RegSetValueExA API calls directly. Fileless uses PowerShell's New-ItemProperty -Path "HKCU:...\Run" cmdlet. LotL uses schtasks /Create /SC ONLOGON /TN "WindowsUpdateCheck" /RL HIGHEST. All achieve logon execution, yet leave different forensic artifacts: registry modifications vs. Task Scheduler entries.

**Network Communication:** Remcos dynamically resolves socket APIs, hence missing WS2\_32.DLL. Fileless uses

`System.Net.WebClient.DownloadString()` with spoofed browser User-Agent. LotL abuses `bitsadmin /transfer`, leveraging legitimate Windows Update infrastructure.

**Anti-Analysis:** Remcos encrypts strings; searches for "Remcos" and "keylog" returned nothing despite being documented capabilities. Fileless checks `$env:COMPUTERNAME` against sandbox patterns like "SANDBOX", "MALWARE", and "VMWARE". LotL destroys evidence through `wevtutil cl Security` and `wevtutil cl System`. These three approaches represent fundamentally different philosophies: hide indicators at compile time (Remcos), detect and avoid analysis environments at runtime (fileless), or destroy forensic evidence after execution completes (LotL).

## 4.5 Indicators of Compromise

**Remcos RAT:** SHA256: 32aa0546c952a980435c113eeb04b6f9a82bf2b260e0b7b261010eaa585d55f1; C2 Server: `http://dsoft1961.narod.ru`; Registry APIs: `RegSetValueExA`, `RegCreateKeyExA` (ADVAPI32.DLL); Compiler: Borland Delphi; Evasion: Missing `WS2_32.DLL/WININET.DLL`.

**Fileless Indicators:** `Invoke-Expression`, `IEX`, `-EncodedCommand`, `-WindowStyle Hidden`; `System.Net.WebClient`; `[Convert]::FromBase64String`.

**LotL Command Patterns:** `certutil -urlcache -split -f [URL]; mshta vbscript:Execute("..."); schtasks /Create /SC ONLOGON; bitsadmin /transfer /upload`.

## 5 Discussion and Challenges

Persistence, while shared in all categories, was implemented differently: Remcos used direct API calls, fileless used PowerShell cmdlets, and LotL used scheduled tasks. Defense evasion was the primary differentiator: Remcos hid via dynamic API loading, fileless operated entirely in memory, and LotL exploited signed binary trust. This finding suggests that effective defense requires layered detection strategies that target each evasion approach rather than relying on any single method.

The C2 URL discovery was notable. Although Remcos used string encryption [8], this sample contained a plaintext network indicator, demonstrating that even obfuscated samples may leak information through incomplete protection. This reinforces the value of thorough string analysis as a standard practice in malware reverse engineering.

Challenges included partial string obfuscation (searches for "Remcos", "keylog", and "CurrentVersion" failed despite these being documented capabilities), asymmetry between binary and pseudocode samples, and time constraints limiting deep behavioral reconstruction. Absent networking imports initially caused confusion; understanding that their absence indicated evasion rather than missing functionality required consulting threat intelligence on documented Remcos behavior. Creating actual functional fileless and LotL samples for direct comparison would raise ethical and legal concerns, hence the pseudocode approach.

Future work could incorporate dynamic sandbox analysis and memory forensics to capture runtime behavior, decrypted strings, and network traffic across multiple samples per category.

**Detection Recommendations:** For RATs like Remcos, network-based detection should monitor connections to known C2 infrastructure. Endpoint detection should alert on processes with registry imports but missing network imports—this pattern suggests dynamic API resolution. File-based detection can leverage compiler artifacts (such as Delphi signatures). For fileless malware, enable PowerShell script block logging (Event ID 4104) and monitor for `Invoke-Expression` combined with `WebClient` or `-EncodedCommand` with long Base64 strings. AMSI integration allows security tools to inspect script content before execution. For LotL attacks, command-line auditing (Event ID 4688) is essential since malicious behavior is in the arguments, not the binary. Key patterns include `certutil -urlcache`, `mshta vbscript:`, `schtasks /Create` with temp directory paths, and `wevtutil cl`. Monitoring parent-child processes is also valuable, as `mshta.exe` spawning `powershell.exe` is suspicious regardless of arguments.

## 6 Concluding Remarks

This project compared three malware categories using Ghidra static analysis: a real Remcos RAT sample, fileless malware techniques, and living-off-the-land attacks. The analysis confirmed that all three categories require persistence mechanisms to maintain access across system reboots, but they achieve evasion through fundamentally different approaches. RATs like Remcos hide capabilities at compile time through dynamic API loading and selective string encryption. Fileless malware avoids detection by architectural choice: never writing to disk means file-based scanning is inherently ineffective. LotL attacks exploit operational trust in Microsoft-signed binaries, requiring behavioral analysis rather than signature matching for detection.

Key findings: (1) Import table analysis is valuable but incomplete. Missing imports may indicate dynamic loading rather than absent capabilities, requiring analysts to consider evasion techniques. (2) String analysis remains productive even against obfuscated samples; the C2 URL recovery demonstrates that protection is often incomplete. (3) Educational pseudocode samples provide a safe method for documenting and comparing attack techniques without handling live malware. (4) Cross-referencing findings against threat intelligence (Elastic, CrowdStrike, MITRE ATT&CK) validates analysis results and provides context for observed behaviors.

The methodology demonstrated here (isolated VM environment, structured Ghidra workflow, MITRE ATT&CK mapping, IOC extraction) could be applied to other malware families for similar classification efforts. The pseudocode samples and Ghidra screenshots are available in the linked repository for replication or extension. Dynamic analysis incorporating runtime behavior, decrypted configuration, and network capture would be the logical next step to complement these static analysis findings.

The broader implications of this analysis extend to enterprise security architectures. Organizations defending against modern threats cannot rely on any single detection layer. Signature-based antivirus misses fileless attacks and struggles against obfuscated RATs. Application whitelisting, while effective against unauthorized executables, provides no protection against LotL attacks using trusted system binaries. Behavioral analysis through EDR solutions offers the most comprehensive coverage, but requires

careful tuning to distinguish malicious from legitimate use of powerful system capabilities like PowerShell and scheduled tasks. The classification framework developed here (distinguishing malware by execution strategy, evasion approach, and persistence mechanism) provides defenders with a mental model for evaluating detection gaps and prioritizing security investments across these fundamentally different threat categories.

## References

- [1] abuse.ch. 2024. *MalwareBazaar: Malware Sample Repository*. <https://bazaar.abuse.ch/> Accessed: 2025-10-19.
- [2] ANY.RUN. 2024. *Remcos RAT Malware Analysis*. <https://any.run/malware-trends/remcos> Accessed: 2025-10-19.
- [3] K. Baker. 2024. *Fileless Malware Explained: How It Works and How to Defend Against It*. <https://www.crowdstrike.com/en-us/cybersecurity-101/malware/fileless-malware/> Accessed: 2025-10-19.
- [4] bytecode77. 2024. *Living-off-the-land: Fileless Attack with Persistence*. <https://github.com/bytecode77/living-off-the-land> Accessed: 2025-10-19.
- [5] Center for Internet Security. 2024. *Living off the Land: Threats Looming From Within Your Network*. <https://www.cisecurity.org/insights/blog/living-off-the-land-threats-looming-from-within> Accessed: 2025-10-19.
- [6] Dimitris. 2024. *Malware Analysis for Beginners: A Step-by-Step Guide*. <https://www.hackthebox.com/blog/malware-analysis-guide> Accessed: 2025-10-19.
- [7] N. Fox. 2023. *How to Use Ghidra to Reverse Engineer Malware*. <https://www.varonis.com/blog/how-to-use-ghidra> Accessed: 2025-10-19.
- [8] C. François and S. Bousseaden. 2024. *Dissecting REMCOS RAT: An In-Depth Analysis of a Widespread 2024 Malware*. <https://www.elastic.co/security-labs/dissecting-remcos-rat-part-one> Accessed: 2025-10-19.
- [9] LOLBAS Project. 2024. *Living Off The Land Binaries, Scripts and Libraries*. <https://lolbas-project.github.io/> Accessed: 2025-10-19.
- [10] MITRE Corporation. 2024. *MITRE ATT&CK Framework: Tactics, Techniques, and Procedures*. <https://attack.mitre.org/> Accessed: 2025-10-19.
- [11] NSA Cybersecurity. 2024. *Ghidra Software Reverse Engineering Framework*. <https://ghidra-sre.org/> Accessed: 2025-10-19.
- [12] M. Sikorski and A. Honig. 2012. *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press.
- [13] VMware. 2024. *VMware Fusion Documentation: Virtual Machine Network Configuration*. <https://docs.vmware.com/en/VMware-Fusion/> Accessed: 2025-10-19.